20-Day Milestone Roadmap: B+ Tree Storage Engine

Building a Disk-Backed, Clustered Index Key-Value Store in Production C++

Target Horizon: 20 Days | **Core Frameworks:** Modern C++ (C++17/20), Low-level Linux/POSIX I/O, Database System Internals

---

This document establishes a highly technical, aggressive, and rewarding 20-day milestone roadmap for implementing a persistent storage engine modeled directly after MySQL's InnoDB clustered indexing. Each day transitions seamlessly between core data structure algorithms and pure low-level systems programming.

Phase 1: Foundations & The Buffer Pool Manager (Days 1–5)

Objective: Abstract the hard drive. Before writing any tree structures, we build the sub-system responsible for reading, writing, and caching fixed-size binary blocks (Pages) from disk into RAM.

Day 1: Disk Layout Schema & Binary File Primitives

- **Learn:** Binary file operations via `std::fstream`, explicit byte seeking offsets (`seekg`, `seekp`), and understanding the file system block allocation rules.
- **Build:** Implement the core `DiskManager` class that opens a single database binary file (e.g., `engine.db`) and provides baseline read/write capabilities for a raw 4096-byte array layout.

Day 2: Fixed-Size Page Design & Memory Mapping Frame Architecture

- **Learn:** Data alignment rules, strict byte sizing across physical boundaries, and avoiding structure padding traps in C++.
- **Build:** Define the fundamental `page_id_t` data type (*uint32_t*). Create a raw `Page` struct containing an internal *4KB* buffer array alongside foundational volatile system metadata metrics (pin count, dirty bit flag).

Day 3: Cache Replacement Policies (LRU / Clock Eviction)

- **Learn:** Memory constraint strategies. How database engines maintain structural reliability when operating under a physical memory limit significantly smaller than the database file size.
- **Build:** Implement a thread-safe `Replacer` class utilizing a custom LRU or Clock eviction tracking algorithm to dictate exactly which data page frames can be safely cleared out from memory when room is required.

Day 4: Buffer Pool Coordinator Logic

- **Learn:** Framing tracking systems, frame-to-page hash mapping structures, and memory-to-disk virtualization mechanics.
- **Build:** Author the initial layout of the `BufferPoolManager`. Build out the `FetchPage(page_id)` routing skeleton logic that intercepts resource requests from higher layers and determines if the targeted data is already cached.

Day 5: Flushing, Pin Management, and Buffer Verification Integration

- **Learn:** Write-back cache constraints and write optimization paradigms. Ensuring pages actively modified or targeted by active queries are locked securely into standard RAM.
- **Build:** Finalize structural implementation of `UnpinPage()` and `FlushPage()` APIs. Tie the `DiskManager`, `Replacer`, and frame layout components into a unified, tested memory management engine.

Phase 2: In-Memory B+ Tree Structure & Core Logic (Days 6–10)

Objective: Code the absolute algorithmic brain. To isolate code bugs, you will first construct the strict topological structure using clean in-memory nodes before binding them to persistent disk page addresses.

Day 6: Node Schema Definitions (Leaf vs. Internal Boundaries)

- **Learn:** Architectural differences between B-Trees and B+ Trees. Internal nodes hold keys/pointers exclusively for navigation; Leaf nodes hold real key/value data records.
- **Build:** Create standard C++ wrapper base classes for `BPlusTreePage`, branching distinctly into separate derived configurations for `InternalNode` and `LeafNode`, complete with maximum element tracking capacities.

Day 7: Binary Search Traversal Implementation (Point Lookups)

- **Learn:** Logarithmic search scaling $O(\log N)$ within discrete arrays. Navigating internal pointer vectors efficiently based on comparative ranges.
- **Build:** Implement the `FindLeafPage()` lookup traversal routine. Write the comprehensive point lookup API `GetValue(key)` that snakes flawlessly from root down to base records.

Day 8: In-Order Insertion Engine (Un-split Boundaries)

- **Learn:** Modern sorting properties inside high-frequency arrays. Fast linear shifting strategies to insert items into pre-sorted data sequences.
- **Build:** Write the baseline `Insert(key, value)` logic. Handle finding the accurate destination leaf node, finding the right relative offset position, and pushing existing records rightward without splitting.

Day 9: Leaf Overflow Mitigation & Structural Node Splitting

- **Learn:** Node boundary mechanics. How a Leaf node dynamically shifts exactly half of its current records over to a newly provisioned peer when its designated maximum array element threshold is breached.
- **Build:** Write the leaf-splitting logic routine. Extract the median key element, allocate a twin Leaf sibling, pass over half the localized data, and push the rising median key into the tracking Parent structure.

Day 10: Cascading Internal Splits & Dynamic Root Upgrades

- **Learn:** Upward structural recursive growth. Mastering the foundational concept that true B+ Trees grow structurally upward from the bottom, causing the Root to split and elevate tree depth.
- **Build:** Implement internal node cascading split handlers. Program safe runtime detection for root level structural overflow, providing logic to spin up pristine top-level master roots dynamically.

Phase 3: Serialization & Slotted-Page Disk Transition (Days 11–15)

Objective: The integration bridge. We destroy the raw RAM memory addresses and convert all algorithmic structural routing logic to use low-level serialized bytes mapped to the Buffer Pool.

Day 11: Complete Variable Migration (Pointer to Page ID Refactor)

- **Learn:** Memory virtualization abstraction. Understanding why direct C++ RAM pointers (`Node*`) cannot reside safely inside binary files saved onto disks.
- **Build:** Perform a total architecture code refactor. Replace all internal child/parent memory pointer references with `page_id_t` integers, modifying all operations to navigate entirely via `BufferPoolManager->FetchPage()`.

Day 12: Binary Slotted-Page Data Layout Serialization

- **Learn:** High-density packing of binary data. Encoding variable sizes using distinct offset slots located cleanly inside fixed 4096-byte arrays.
- **Build:** Design a robust binary structural encoding function. Implement `SerializePage()` using raw `std::memcpy` to write key arrays, payload arrays, and precise structural page headers directly to raw byte blocks.

Day 13: Deserialization Rehydration Routine

- **Learn:** C++ Type casting models and binary conversions (`reinterpret_cast`). Translating flat disk-backed binary buffers directly back into structured, usable in-memory nodes.
- **Build:** Code the corresponding mirror function `DeserializePage()`. Ensure that when raw data blocks enter RAM via the Buffer Pool, they instantly manifest as highly functional Node structures.

Day 14: Sequential Link Implementation & Clustered Range Scans

- **Learn:** Clustered indexing scans. How linking leaf pages sequentially enables incredibly fast range scan queries without traversing upper tree infrastructure.
- **Build:** Implement a `next_page_id` tracking field across the base leaf headers. Build a comprehensive `Scan(start_key, end_key)` iterator function that quickly hops across discrete disk blocks for data collection.

Day 15: Critical Resource Concurrency & Structural Latching

- **Learn:** Thread safety concepts and avoid deadlock states. The theory of Latch Crabbing: locking a target child node before releasing the corresponding parent safety handle.
- **Build:** Inject robust thread safety constraints into the core traversal pathways utilizing `std::shared_mutex` to cleanly isolate concurrently running read and write operations.

Phase 4: Tombstoning, Resiliency, & Testing (Days 16–20)

Objective: Solidify into a production-grade utility. We introduce highly performant deletion logic, structure stress-testing tools, and build out a functional command-line interaction layer.

Day 16: Logical Deletion via Tombstone Tracking

- **Learn:** Real-world performance engineering tradeoffs. Realizing physical page merging and shifting during deletion operations introduces high risk; logical tombstones match production performance paths.
- **Build:** Build a high-performance `DeleteKey(key)` operation. Implement a bitmap or byte indicator flag on the records to signify deletion, ensuring point lookups and scans treat marked items as absent.

Day 17: High-Volume Random Data Stress Validation

- **Learn:** Automated fuzzing paradigms, validation strategies, and invariant verification testing.
- **Build:** Build a stress-testing engine script that generates thousands of random keys. Fire massive volumes of keys at the storage engine, checking structural integrity after massive page splitting.

Day 18: Performance Profiling & Buffer Cache Optimization Metrics

- **Learn:** Identifying memory bottlenecks. Tracking hit-to-miss ratios inside the buffer pool cache to evaluate real-world cache strategy efficiency.
- **Build:** Integrate explicit profiling metrics tracking file read/write metrics. Test performance across various pool scales to watch operations drop as cache efficiency climbs.

Day 19: Binary File Recovery & Boundary Cleanups

- **Learn:** Database crash-persistence basics. Validating that restarting a runtime process instantly restores the entire system state back to active operations by re-reading the root block identifier.
- **Build:** Code a deterministic master block schema mapping standard boot tracks. Hardcode the Root Page ID at byte zero of the file layout to ensure clean reboots parse data paths flawlessly.

Day 20: Interactive REPL Engine Console Deployment

- **Learn:** Interface design principles, system evaluation methods, and showcasing modular backend engines cleanly.
- **Build:** Construct a clean, human-navigable database CLI loop terminal (REPL). Expose commands like `put [k] [v]`, `get [k]`, and `scan [start] [end]` to drive your custom persistent engine.